SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO -2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Design Task

Weightage: 20%

Code of Conduct:

I K. Lahari (192325118) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K. Lahari

QUESTION 1: RESPONSIVE NAVIGATION BAR DESIGN (4 MARKS)

Problem statement: An e-commerce website requires a navigation bar that adapts to desktop, tablet and mobile devices.

(a) Identify the key components required in an e-commerce navigation bar.

A useful e-commerce navigation bar should contain the website logo, a product search field, category links, an account link, a shopping-cart indicator and a menu button for small screens. The logo helps users recognise the brand. Search allows users to locate products quickly, while category links group products into sections such as Electronics, Fashion and Home & Living. Account and cart links provide quick access to user and purchase-related features.

(b) Design a responsive navigation bar using appropriate CSS techniques.

The NovaMart interface uses Flexbox in the header to arrange the logo, search field and account/cart actions along one row. The search area uses flex: 1 so that it can occupy the available space without pushing other items off-screen. CSS padding, borders, colours and hover states provide consistent spacing and visual feedback. The category links are displayed horizontally on larger screens.

(c) Demonstrate how the navigation adapts to smaller screens.

A media query is used when the viewport width becomes smaller. The search field moves to a separate row, the horizontal category navigation is hidden and a hamburger button becomes visible. When the user clicks the hamburger button, JavaScript adds or removes a CSS class to display the navigation links. The aria-expanded attribute is updated to communicate whether the menu is open or closed.

Important implementation code

```
/* Desktop navigation */
header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 22px;
}
nav {
  display: flex;
  justify-content: center;
  gap: 30px;
}

/* Mobile navigation */
@media (max-width: 850px) {
  .menu { display: block; }
```

```
  nav { display: none; flex-direction: column; }
  nav.open { display: flex; }
}
toggle.addEventListener("click", () => {
  const open = nav.classList.toggle("open");
  toggle.setAttribute("aria-expanded", open);
});
```

(d) Evaluate usability and suggest improvements.

The navigation is easy to scan because the most important actions are visible in the header. The mobile menu saves screen space and prevents links from becoming crowded. Search and category links support product discovery. For further improvement, the search field could show matching products while typing, the cart could open a dedicated cart page, and the menu should be tested with keyboard navigation, screen readers and different mobile widths. In this demonstration, search displays a message; it is not connected to a product database.

Output and observation

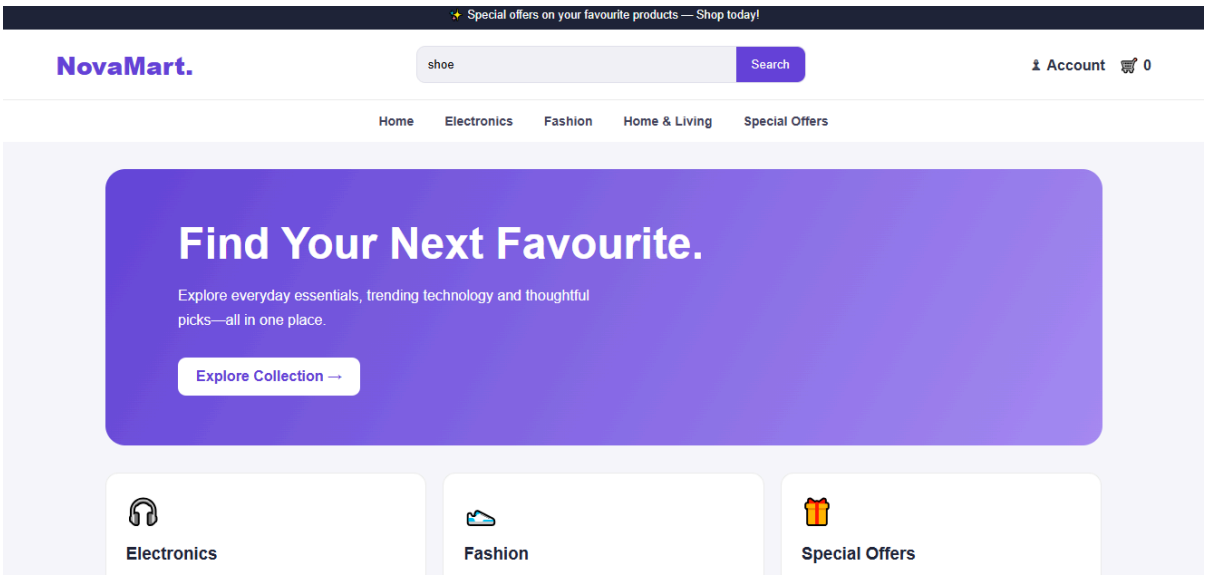


Figure 1. NovaMart home page showing the logo, search field, account, cart and category navigation.

Observation: The page presents a horizontal navigation on a wide screen and uses a hamburger menu at smaller widths. This satisfies the responsive navigation requirement.

QUESTION 2: PRODUCT HOVER EFFECT (4 MARKS)

Problem statement: An e-commerce website wants to improve user interaction by applying visual effects to product items.

(a) Identify suitable CSS properties for hover effects.

The suitable properties include transform, box-shadow, transition, border-radius and opacity. The transform property can lift a card or zoom an image. Box-shadow creates depth and helps the hovered item stand out. Transition controls how smoothly the changes occur. Border-radius gives the product cards rounded corners.

(b) Suggest an appropriate CSS animation or transition.

A short transition lasting approximately 0.2 to 0.4 seconds is appropriate for a product card. When the pointer moves over the card, it moves slightly upward and its shadow becomes stronger. The product image zooms in a small amount. The transition should be subtle and should not cause nearby cards to jump or move.

(c) Design a hover effect for a product card.

The NovaMart product gallery displays a product image, category, product name, star rating, price, optional original price, sale badge and Add to Cart button. The hover effect is applied to the complete card and its image. JavaScript updates the cart counter when a user presses Add to Cart.

Important implementation code

```
.card {
  transition: transform 0.28s ease,
              box-shadow 0.28s ease;
}
.card:hover {
  transform: translateY(-8px);
  box-shadow: 0 16px 30px #6246d930;
}
.photo img {
  transition: transform 0.4s ease;
}
.card:hover img {
  transform: scale(1.07);
}
let count = 0;
document.querySelectorAll(".add").forEach(button => {
  button.addEventListener("click", () => {
    count++;
    document.getElementById("count").textContent = count;
  });
});
```

(d) Evaluate performance and user experience.

The design uses CSS transitions instead of repeatedly running a complex animation, so the effect is lightweight. Images have alternative text, and the reduced-motion media query disables transitions for users who prefer less motion. The Add to Cart action gives immediate feedback. To improve a production site, product images should be compressed, prices and ratings should be accurate, and the cart should store selected products rather than only counting clicks.

Output and observation

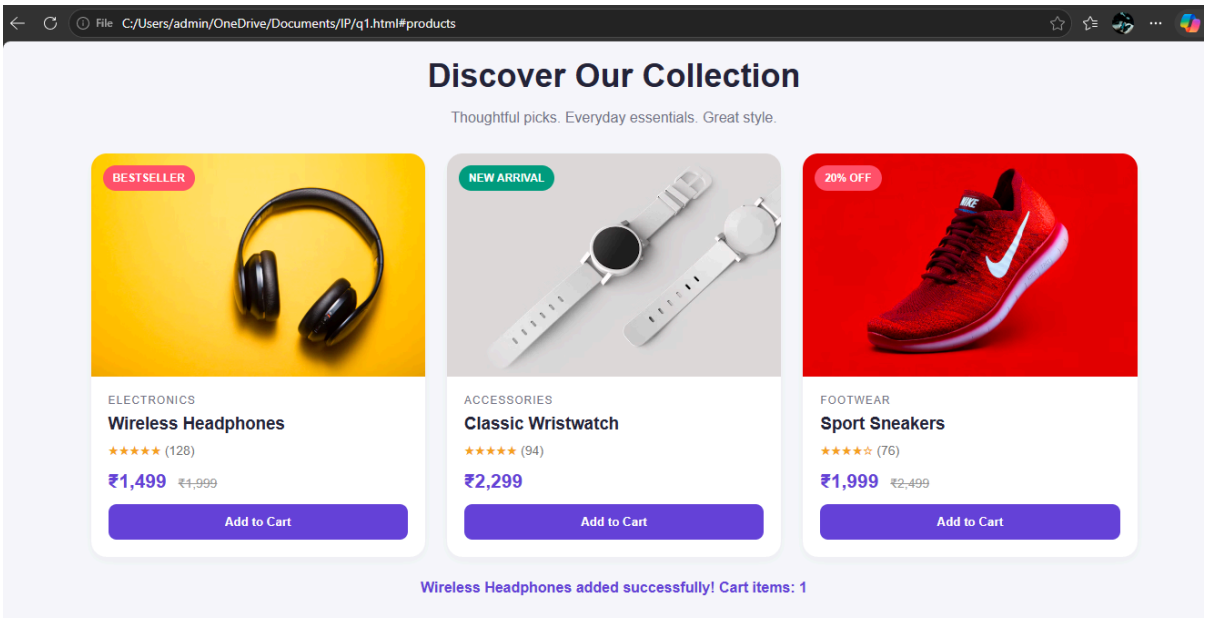


Figure 2. Product gallery showing images, sale badges, ratings, prices and Add to Cart controls.

Observation: The product cards provide visual feedback on hover, while the cart counter confirms the user's action. The design makes product browsing more engaging without adding a complicated animation.

QUESTION 3: LAYOUT SELECTION — FLEXBOX VS GRID (4 MARKS)

Problem statement: A dashboard layout must be designed for a web application.

(a) Identify dashboard layout requirements.

A dashboard commonly contains a sidebar, top header, summary cards, charts, category summaries and a table of recent orders. These components need consistent alignment, spacing and grouping. The layout must also adapt when the viewport changes from desktop to tablet or mobile.

(b) Compare Flexbox and Grid based on layout needs.

Flexbox is mainly a one-dimensional layout system. It is suitable when items need to be arranged in a row or a column, such as aligning the logo and account details in a header. CSS Grid is a two-dimensional system that manages rows and columns together. It is suitable for arranging a sidebar beside the main content and for creating a grid of statistic cards. Both techniques can be used together.

(c) Choose the most suitable technique and justify the choice.

CSS Grid is the most suitable primary technique for this dashboard because the overall page has two-dimensional structure: a sidebar column and a main content area. Grid also arranges the statistic cards and dashboard panels. Flexbox is used inside individual components to align items horizontally or vertically. This combination uses each layout technique for the task it handles best.

Important implementation code

```
.layout {
  display: grid;
  grid-template-columns: 230px minmax(0, 1fr);
}
.stats {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 16px;
}
.topbar {
  display: flex;
  align-items: center;
  justify-content: space-between;
}
@media (max-width: 1050px) {
  .stats { grid-template-columns: repeat(2, 1fr); }
  .content { grid-template-columns: 1fr; }
}
@media (max-width: 650px) {
  .layout { grid-template-columns: 1fr; }
}
```

(d) Evaluate scalability and responsiveness.

The Grid structure is scalable because additional statistic cards and panels can be added without manually positioning every element. Media queries reduce the number of columns on smaller screens. The sidebar changes into a horizontal navigation area on mobile, and the recent-orders table can scroll horizontally if the screen is too narrow. For a real dashboard, the sample revenue, order and customer values should be loaded from a reliable data source.

Output and observation

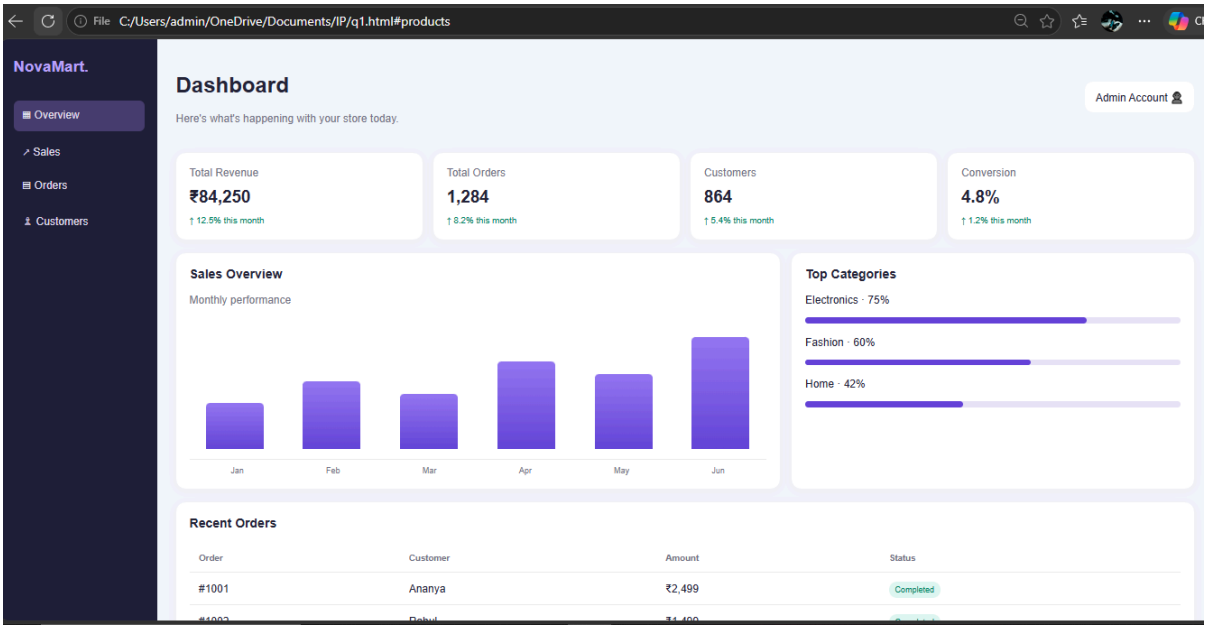


Figure 3. Dashboard output showing sidebar, summary cards, sales chart, category progress and recent orders.

Observation: Grid organises the main dashboard into clear sections, while Flexbox aligns items within the header. The media queries allow the layout to adapt to smaller screens.

QUESTION 4: MOBILE-FIRST BLOG LAYOUT (4 MARKS)

Problem statement: A blog page must be designed using a mobile-first approach.

(a) Identify the key sections of a blog layout.

The main sections are a site header, navigation links, featured article, latest-stories section, individual article cards, an About section and a footer. Each article card contains an image, category, title, short description and a link to read more.

(b) Design a mobile-first layout using CSS.

A mobile-first layout begins with styles for small screens. The content is placed in one column so that users can read each article without horizontal scrolling. Images fill the width of their containers and use object-fit: cover. The design uses a readable font size, sufficient line spacing and space between article cards.

Important implementation code

```
/* Default styles are for mobile screens */
.articles {
  display: grid;
  grid-template-columns: 1fr;
  gap: 20px;
}
.featured {
  display: grid;
  grid-template-columns: 1fr;
  gap: 24px;
}
```

(c) Demonstrate how the layout scales for larger screens.

Media queries progressively add columns as more screen space becomes available. At a tablet width, the featured article becomes a two-column layout and the article cards use two columns. At a wider desktop width, the article list expands to three columns. This approach avoids designing for desktop first and then trying to squeeze the same layout onto a phone.

```
@media (min-width: 650px) {
  .featured { grid-template-columns: 1.05fr 1fr; }
  .articles { grid-template-columns: repeat(2, 1fr); }
}
@media (min-width: 950px) {
  .articles { grid-template-columns: repeat(3, 1fr); }
}
```

(d) Evaluate accessibility and readability.

Semantic HTML elements such as header, nav, main, section and article make the page structure meaningful. Descriptive alternative text helps users who cannot see images. Clear heading levels, descriptive link labels and comfortable line spacing improve readability.

Further improvements include visible keyboard-focus styles, checking text/background contrast and ensuring the layout remains usable when images do not load.

Output and observation

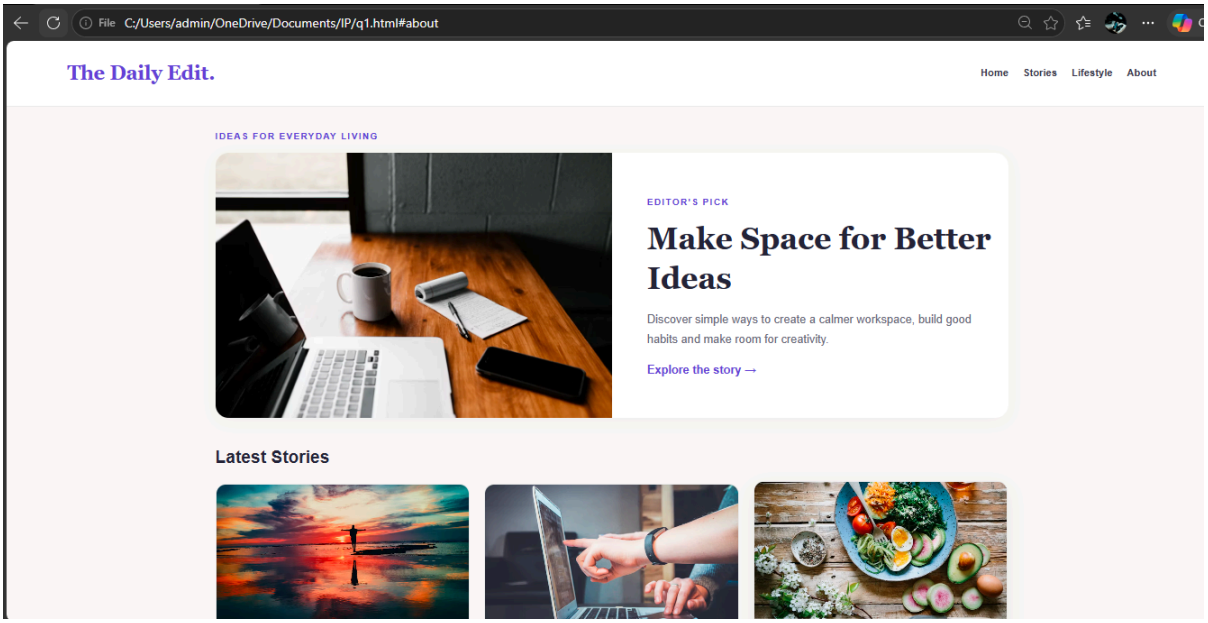


Figure 4. The Daily Edit blog homepage showing the featured story and latest articles.

Observation: The featured story is visually prominent, while article cards provide a consistent reading experience. The responsive grid changes from one column on mobile to two columns on tablets and three columns on desktop.

QUESTION 5: JAVASCRIPT EVENT HANDLING FOR DYNAMIC MENU (4 MARKS)

Problem statement: A website includes a dynamic menu that responds to user interactions.

(a) Identify appropriate JavaScript events.

The click event allows users to open or close the Products menu. Mouseenter and mouseleave events support hover interaction on devices with a mouse. The keydown event handles the Escape key. A click event on the document can detect when the user clicks outside the menu and close it.

(b) Design event handling logic for menu toggle functionality.

The menu uses a function called setMenu(open). The function adds or removes the open class on the dropdown and updates the aria-expanded attribute. This keeps the visual state and accessibility state in sync. A single function is reused for different events to avoid duplicating the same code.

Important implementation code

```
function setMenu(open) {
  dropdown.classList.toggle("open", open);
  button.setAttribute("aria-expanded", open);
}
button.addEventListener("click", function () {
  setMenu(!dropdown.classList.contains("open"));
});
```

(c) Implement interaction for click and hover actions.

When the Products button is clicked, the submenu opens or closes. On desktop devices that support hover, moving the pointer over the menu opens it and moving away closes it. Choosing a category closes the submenu. Clicking outside the dropdown also closes it. Pressing Escape closes the menu and returns keyboard focus to the Products button.

```
document.addEventListener("keydown", function(event) {
  if (event.key === "Escape") {
    setMenu(false);
    button.focus();
  }
});
document.addEventListener("click", function(event) {
  if (!dropdown.contains(event.target)) {
    setMenu(false);
  }
});
```

(d) Evaluate performance and suggest improvements.

The menu uses a small amount of JavaScript and does not require an external library, so it is lightweight. Event listeners are attached to relevant elements, and the submenu is hidden when not in use. Keyboard support and aria-expanded improve accessibility. Further testing should cover touch devices, keyboard movement between submenu links, browser zoom and narrow screen sizes. Hover should not be the only way to access the menu because touchscreens do not always support hover.

Output and observation

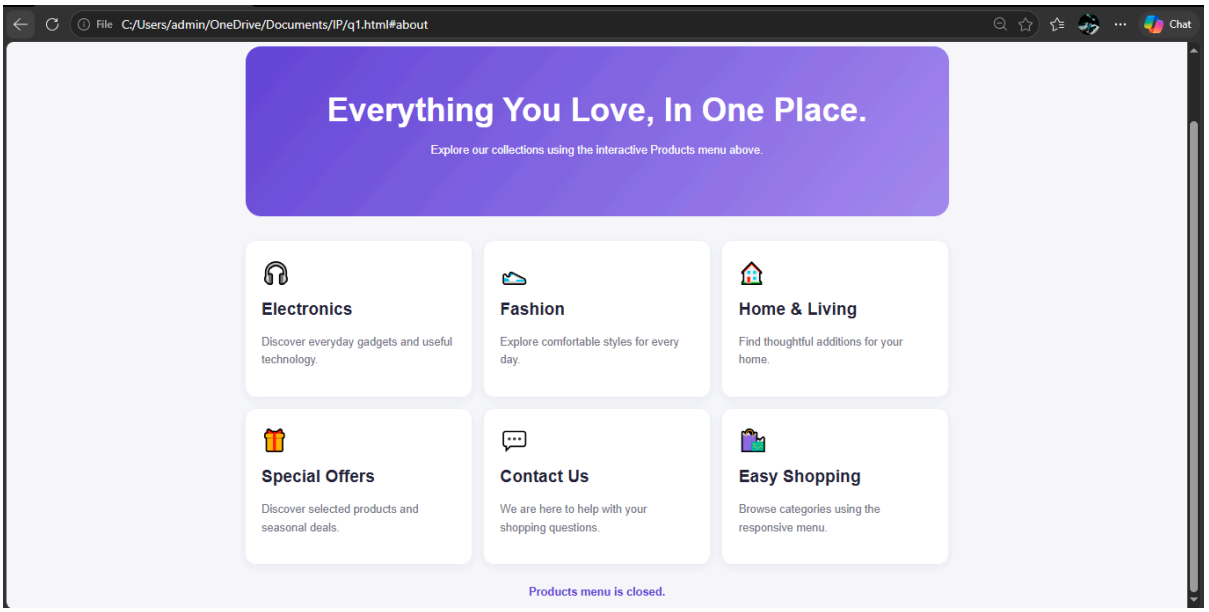


Figure 5. NovaMart interactive menu page showing the category cards and menu status.

Observation: The menu responds to clicks, hover and keyboard input. The status message makes the current open/closed state visible during demonstration.